MODULE 4 · LEVEL 300

Development: Building with AI

From a buildable design to working software - implement, refactor, review and secure, with GitHub Copilot

Day 2 · 90 minutes · Builds on Module 3 (the design) · Novnex

Where we are, and today's shape

approx 2 min

Module 3 produced a buildable design. Today we turn it into working software - and this is where Copilot is strongest.

Framing

Why development is Copilot's home turf, and the map.

~16 min

Development with Copilot

Implement, understand, refactor, delegate, review, secure, debug.

~50 min

Apply

By track, anti-patterns, ways of working, the hand-off, the lab.

~24 min

The guardrails still hold: ground the context, review everything - and here, review carries the most weight of any phase.

Learning objectives

approx 2 min

By the end of this module you will be able to:

Build from the design

Implement features to your contracts and ADRs with local Agent mode in VS Code.

Use the full toolset

Match completions, chat and agents to the task - flow, understanding, or whole features.

Review & secure

Review AI and agent code critically, and catch vulnerabilities as you write.

Stay accountable

Keep humans owning what merges - small diffs, tests, no secrets in prompts.

Development is Copilot's home turf

approx 4 min

The biggest gains

- **Routine work:**
- boilerplate, wiring, glue code - drafted in seconds.
- **Unfamiliar ground:**
- a new API, language or codebase becomes approachable.
- **Whole features:**
- build well-scoped issues with local Agent mode.
- **Momentum:**
- stay in flow instead of context-switching to search.

The biggest responsibility

- More code, faster, means more code to review
- Suggestions can be wrong, insecure or licence-tainted
- Accepting code you can't explain is how bugs ship
- So the skill shifts from writing to judging

Where AI helps across development

approx 4 min

Implement

Build features to the design, contract and ADRs.

Complete

Inline suggestions that keep you in flow.

Understand

Explain unfamiliar and legacy code fast.

Refactor

Improve structure safely, in small steps.

Review

A fast first pass on every change.

Debug

Explain errors and propose fixes.

We'll take these in order. Development is deep - so today we build, and Module 5 proves it works.

Match the mode to the task

approx 3 min

1

Complete

A line or a block. Tab to accept.

Small & in-flow

2

Chat / Ask

Understand, fix, or plan changes.

Medium & guided

3

Agent

A whole feature or chore, end to end.

Large & delegated

Most features touch all three. The skill is choosing - and reviewing more as you delegate more.

PART 2

Development with Copilot

Implement, understand, refactor, delegate, review, secure and debug - with real examples.

approx 50 min

Implement straight from the design

approx 5 min

Feed Copilot the contract and the ADR, and let agent mode build to them - not to a vague description.

Agent mode

```
Implement POST /password-reset per the
OpenAPI contract and ADR 007
(single-use token, 30-min expiry, generic 202).
Add validation and tests; keep rate limit TBC.
# edits handler, service, routes
$ runs tests -> 1 failing -> fixed
```

Why it lands

- **Builds to the contract:**
- the API, the ER model, the ADR are the spec.
- **Multi-file & self-healing:**
- edits, runs tests, fixes failures.
- **Design pays off:**
- a clear design is why the output is correct.
- **You review the diff:**
- accept deliberately, not blindly.

Stay in flow with completions

approx 4 min

Comment to code

```
# return the customer's active tickets,
# most recent first
def active_tickets(customer_id):
    ...  # Copilot completes; Tab to accept
```

Get better suggestions

- **Write the intent first:**
- a clear comment or signature steers it.
- **Good names help:**
- it reads your context to predict.
- **Keep files open:**
- nearby code is its context.
- **Accept, then read:**
- never Tab past code you don't follow.

Chat is your pair programmer

approx 4 min

/explain

Understand a function, error or diff in plain language.

/fix

Propose a fix for a selected problem or failing test.

#codebase

Ask about the whole project, not just the open file.

Ask anything

'Why is this slow?' 'Is this thread-safe?' 'Simplify this.'

Understand unfamiliar & legacy code

approx 4 min

Onboard fast

- **Explain it:**
- 'walk me through this module and its side effects.'
- **Map the flow:**
- ask for a call graph or a sequence diagram.
- **Find the entry points:**
- where does this behaviour start?
- **Document as you learn:**
- generate docstrings and notes.

Stay careful

- On unfamiliar code, verify its explanation against reality
- It can misread intent in gnarly legacy logic
- Use it to form a hypothesis, then confirm by reading
- Great for ramp-up; not a substitute for understanding

Refactor safely, in small steps

approx 4 min

Guided refactor

```
# select the function, then ask:
"Extract validation into a helper and
 add early returns; keep behaviour."
- nested if/else, 40 lines
+ validate() helper + guard clauses
$ tests still green
```

Refactor without fear

- **Tests are the guardrail:**
- refactor behind green tests.
- **Small steps:**
- one change at a time, easy to review.
- **Name the goal:**
- 'reduce coupling', 'remove duplication'.
- **Read the diff:**
- confirm behaviour really is unchanged.

Build whole features with local Agent mode

approx 5 min

For well-scoped work, attach the issue and let local Agent mode draft a change you review in VS Code.

Issue -> local diff

```
Issue #142: implement change-email flow
Attach issue + design
-> local branch change-email
   edits + tests in VS Code
   local diff awaiting review
```

Scope it well

- Best for well-scoped, testable issues
- Clear acceptance criteria = better results
- Guardrails: exact files, fixed tests, small diff
- Copilot drafts; a human reviews and signs off

Review AI and local Agent code

approx 4 min

More generated code means review is the bottleneck - let Copilot go first, then keep a human on the hand-off.

Copilot review comment

```
service/reset.py  line 31
[ MEDIUM ] token compared with ==
use a constant-time comparison to avoid
a timing side-channel.
[ Suggested change ]
```

Two passes

- **Copilot first:**
- catches the obvious; severity-labelled.
- **Human second:**
- intent, design fit, edge cases.
- **Small diffs:**
- make both reviews actually possible.

Treat an Agent-generated change like a capable junior's draft - read every line before sign-off.

Secure coding as you write

approx 5 min

Catch it early

```
- query = "SELECT * FROM users WHERE"
-         + " email='" + email + "'"
+ query = "... WHERE email = %s"
+ db.execute(query, [email])
# ask Copilot: 'any security issues here?'
```

Security in the loop

- **Ask as you go:**
- 'review this for security' before you commit.
- **Local verification:**
- run targeted tests and inspect the diff.
- **Never trust by default:**
- AI can reproduce insecure patterns.

Secure-by-habit: prompt for security, run tests, inspect the diff, and never keep a security finding unread. Deep security ops is Module 7.

Debug with Copilot

approx 4 min

From stack trace to fix

```
KeyError: 'email'  (reset.py:22)
> paste the error, ask 'why?'
Copilot: the payload may omit 'email';
validate input and return a 400.
> /fix  -> guard clause + test
```

A faster loop

- **Explain the error:**
- plain-language cause, not just the trace.
- **Hypothesise:**
- it suggests likely culprits to check.
- **Fix + test:**
- turn the bug into a regression test.
- **You confirm:**
- reproduce, then verify the fix holds.

Ground development in your conventions

approx 4 min

Attached local context makes Copilot write code that looks like your team's - not generic code.

Attach with Add Context

```
- Use our repo layout and error types.
- Prefer composition; small functions.
- All new code in TypeScript; strict mode.
- Write a test with every change.
# house rules, standards, patterns, ADRs
```

What it buys you

- **House style:**
- suggestions follow your conventions.
- **Fewer nits:**
- less review time on formatting and patterns.
- **Consistency:**
- the task gets the right local context.
- **Task-scoped:**
- attachments guide this VS Code Chat/Agent session.

Live demo: design to working feature

approx 5 min

Watch it come together

- Agent mode implements the feature to the contract
- It runs the tests and fixes its own failure
- Copilot review flags an issue; we apply the fix
- We ask for security review before committing
- We reject one suggestion - on purpose

Flow

- **1.**
- Point agent at the contract.
- **2.**
- Watch multi-file edit + tests.
- **3.**
- Run Copilot review.
- **4.**
- Ask for a security pass.
- **5.**
- Review - accept & reject.

Responsible development with AI

approx 3 min

Unreviewed output	Accepting code nobody read.	**Guard:** Small diffs; a human reviews every generated change.
Secrets in prompts	Pasting keys or customer data.	**Guard:** Never paste secrets; use content exclusion; follow policy.
Invention & gaps	Guessing TBC values or missing interfaces.	**Guard:** Keep TBC and absent seams explicit; assign owners.
No tests	Shipping unverified code.	**Guard:** Every change carries a test; green is the minimum bar.

PART 3

Apply it

By track, the anti-patterns, team habits, the hand-off to testing, and your lab.

approx 24 min

Development with Copilot - by track

approx 4 min

Web / Drupal

PHP modules, hooks and Twig; JS front-end; agent-built features to the design.

Data & SQL

Natural-language to SQL, transformation scripts, and explaining complex queries.

Power BI

DAX measures and Power Query (M); explain and document the model.

RPA

Automation scripts and expressions; refactor and document existing bots.

Anti-patterns to avoid

approx 3 min

Don't

- Accept code you can't explain
- Merge huge AI-generated diffs unread
- Assume it's secure or licence-clean
- Let velocity outrun your review capacity
- Hand the agent a vague, sprawling task

Do

- Review every change - AI and human alike
- Keep diffs small enough to actually read
- Write a test with each change
- Ground Copilot in your conventions
- Delegate only well-scoped, testable work

Ways of working for teams

approx 3 min

Small changes

Keep changes reviewable; the review is the real gate.

Shared instructions

Attached standards, patterns and ADRs keep style consistent.

Review capacity

Plan for it - generation is cheap, review is the constraint.

Measure

Track cycle time, review time and defect escape - your own baseline.

The hand-off to testing

approx 3 min

'Done' in development isn't 'it runs' - it's 'it's proven'. Tests are the gate into the next phase.

Meets the criteria

Every acceptance criterion implemented.

Tests with it

Unit tests written and green.

Reviewed & secure

Human-approved; security pass done.

Clean PR

Small, documented, criteria linked.

Module 5 goes deep on making that test suite trustworthy - today we just make sure it exists.

Your lab: build a feature with AI

approx 2 min

Lab 4 (this afternoon) - on your own track

- Take a well-scoped issue for your track (from your design in Lab 3)
- Implement it with local Agent mode in VS Code
- Run a local Chat review and security pass; write a test
- Review critically - accept the good, reject and fix the weak

Role groups; pair up. The goal is one reviewed, tested, build-ready change - not volume.

Discussion & reflection

approx 4 min

Talk to the person next to you - three minutes:

- Which coding task would you hand to an agent first - and which would you still write yourself?
- How will your team keep review from becoming the bottleneck as generation speeds up?

Delegation	Review capacity	Security	Standards	Skill growth

We'll take a few answers before wrapping up.

Key takeaways

- Copilot is strongest here: implement, understand, refactor, review, debug.
- Build to the design - contracts and ADRs make agent output correct.
- Match the mode to the task; review more as you automate more.
- Secure and test as you write; ground Copilot in your conventions.
- Generation is cheap; judgement is the job - review every line.

What's next

Module 5

Testing

then

Lab 4 this afternoon

Questions?